

PREDICTOR DE DEMANDA PARA REDES ELÉCTRICAS

CARLA ÁLVAREZ GARCÍA

IES SAN ANDRÉS CURSO DE ESPECIALIZACIÓN IA Y BIG DATA

Contenido

Fase I: Comprensión del Negocio (Business Understanding).....2

 Criterios de Éxito del Proyecto.....2

Fase II: Comprensión de los Datos (Data Understanding)2

Fase III: Preparación de los Datos (Data Preparation)3

 3.1 Capa Bronce: Ingesta Híbrida (AWS y Local).....3

 3.2 Capa Plata: Limpieza con Spark.....3

 3.3 Capa Oro: Cruce, Feature Engineering y Gestión de Nulos3

Fase VI: Despliegue (Deployment)4

Diccionario de Datos (Capa Oro)5

Fase I: Comprensión del Negocio

Hoy en día, el sector eléctrico está cambiando muchísimo con todo el tema de las renovables (solar, eólica). El problema principal que tienen estas energías es que dependen del clima, así que la generación pega muchos saltos. Para que la red eléctrica no se caiga y no haya apagones, hay que cuadrar muy bien lo que se genera con lo que se consume en todo momento.

El objetivo de este proyecto es montar toda la arquitectura de datos desde cero para que luego sirva de base a un modelo de Machine Learning. El sistema tiene que juntar automáticamente el histórico de consumo en España, el clima de cada día y si es festivo o no para poder encontrar los patrones.

Criterios de Éxito del Proyecto

- **Fiabilidad y Control de Errores:** Tiene que conectar bien con las APIs de AWS y descargas locales, utilizando bloques try-except para que el pipeline no se rompa y deje logs claros si falla una conexión. Que si ejecuto el código de Spark en local varias veces seguidas por error, no se dupliquen los datos ni se corrompa el histórico.
- **Datos limpios (Capa Oro):** Crear un dataset final optimizado, gestionando conscientemente la latencia de las distintas fuentes, listo para meterlo a algoritmos como Random Forest o XGBoost.

Fase II: Comprensión de los Datos

Para predecir bien la demanda, he montado la arquitectura consumiendo tres fuentes de datos diferentes, cada una con su formato y forma de actualizarse:

1. **Demanda Eléctrica (Variable Objetivo):** Lo saco de la API pública de REE (<https://apidatos.ree.es/es/datos/balance/balance-electrico>). Me da los MW consumidos por día. Viene en un JSON multilínea bastante anidado que hay que aplanar.
2. **Clima (AEMET):** Hago web scraping en <https://datosclima.es/Aemet2013/DescargaDatos.html>. Son archivos Excel (.xls) diarios comprimidos en .rar desde 2013 con las temperaturas y lluvia. Luego hago la media nacional.
3. **Festivos (Calendarific):** Consumo la API <https://calendarific.com/api/v2/holidays> para saber qué días son fiesta en España, porque la demanda cae en picado cuando la industria cierra.

Problemas encontrados con la calidad de los datos:

Al revisar los Excels del clima vi que algunos venían corruptos, con nombres raros, o las cabeceras estaban sucias. Además, en la API de REE a veces faltaban datos

sueños justo en los días que se cambia la hora, así que hubo que controlarlo para no fastidiar las fechas.

Fase III: Preparación de los Datos

Esta parte es el núcleo del proyecto. He usado una Arquitectura Medallón para separar la ingesta en bruto, la limpieza y el cruce final de las variables.

3.1 Capa Bronce: Ingesta Híbrida (AWS y Local)

Aquí guardo los datos en crudo en el bucket de S3 en AWS (cag-proyecto-demanda-electrica-bd). Para automatizarlo y tener un seguimiento de fallos lo hice de dos formas:

- **Demanda y Festivos (Nube):** Hice unas funciones Lambda en AWS usando la librería `urllib3`. Para la demanda, el código calcula el día exacto de ayer (`datetime.now() - timedelta(days=1)`) para hacer descargas incrementales de forma autónoma. Para los festivos, itera dinámicamente hasta el año actual. Todo el código está envuelto en bloques `try-except` que devuelven el código de error HTTP exacto si la API falla, asegurando la estabilidad del flujo. Todo se guarda particionado (ej. `bronze/demanda/year=2026/month=05/`).
- **Clima (Local):** Como los Excels vienen comprimidos en `.rar`, usé `BeautifulSoup` para scrapear los links de descarga. El script usa `calendar.monthrange` para saber cuántos días exactos tiene un mes y detectar si la descarga está incompleta. Para extraerlos, instalé por consola el paquete `unar` (`apt-get install -y --no-install-recommends unar`) y ejecuté subprocesos forzados.

3.2 Capa Plata: Limpieza con Spark

Una vez en S3, uso Apache Spark conectándome al clúster master (`spark://spark-master:7077`) para procesar los datos en memoria. Spark lee los archivos y hace estas limpiezas:

- **Lectura multilínea de JSONs:** Tuve que forzar a Spark con la opción `.option("multiline", "true")` porque la API de REE devuelve bloques anidados.
- **Limpieza de Excels corruptos:** Como los xls de la AEMET tenían cabeceras rotas, usé `Pandas` con el motor `xlrd` para saltarme las 4 primeras filas (`skiprows=4`) antes de pasarlo a `DataFrames` de Spark.
- **Guardar en Parquet:** Todo se guarda localmente en carpetas separadas usando `.coalesce(1)` para que quede un solo archivo eficiente en modo `overwrite`. Esto garantiza la idempotencia del script y elimina duplicados.

3.3 Capa Oro: Cruce, Feature Engineering y Gestión de Nulos

Este es el dataset final para el modelo predictivo. Spark hace un `Left Join` usando la fecha para juntar la demanda con el clima y los festivos.

Justificación Técnica para la Defensa: La Latencia de Datos y los Nulos en el Clima.

En el dataset final de la Capa Oro, los últimos días de registro tienen datos de consumo eléctrico, pero las columnas del clima (temperaturas y precipitaciones) aparecen con valores NULL. Esto es un comportamiento totalmente intencionado para gestionar la latencia de datos. La API de REE es un sistema crítico que escupe datos en tiempo real, mientras que la AEMET tarda varios días en consolidar y publicar sus Excels mensuales.

Al utilizar un LEFT JOIN (manteniendo la demanda a la izquierda), el pipeline asegura que no perdemos nuestra variable objetivo de los días más recientes. Si hubiese usado un INNER JOIN, habríamos borrado días enteros de consumo eléctrico válido solo porque faltaba el dato del clima. En un entorno real de IA, esos valores nulos del clima se imputarán posteriormente en la fase de modelado utilizando las previsiones meteorológicas.

Además de juntar las tablas, generé estas variables (Feature Engineering):

- Festivos a 0 y 1: Si un día no cruzaba con la tabla de festivos, lo pasé a 0 (laborable) y dejé 1 para los festivos.
- Fechas separadas: Saqué columnas sueltas para dia_semana, mes, anio, trimestre y si cae en fin_de_semana.
- Rango térmico y Lag: Calculé el rango_termico restando la máxima menos la mínima. También usé funciones de ventana analítica (Window.partitionBy) para crear el demanda_lag_7 (la demanda que hubo exactamente hace una semana), silenciando los warnings nativos de Spark al procesarlo.

Fase VI: Despliegue (Deployment)

Para cumplir con los resultados de aprendizaje de Sistemas, monté un Dashboard web interactivo instalando las librerías Altair.

Para que el entorno no dé problemas de librerías y tener un aislamiento perfecto, metí la aplicación entera dentro de un contenedor Docker mapeando los puertos correspondientes de forma local. Usé @st.cache_data para que Streamlit lea el archivo Parquet de la Capa Oro una sola vez, optimizando enormemente el rendimiento del Dashboard.

Los cuatro gráficos de Altair (línea temporal, dispersión, barras y mapa de calor estacional) son puramente interactivos. Permiten investigar el efecto en forma de "U" de las temperaturas frente a la demanda o cómo decae brutalmente la carga industrial durante los fines de semana.

Diccionario de Datos (Capa Oro)

Esta tabla es el contrato de datos técnico que documenta las columnas guardadas en dataset_predictivo.parquet.

Columna	Tipo de Dato	Explicación	Origen
fecha	DateType	Clave principal para cruzar los datos. Formato YYYY-MM-DD.	API REE / Clima
demanda	FloatType	Los MW consumidos en total ese día (lo que queremos predecir).	API REE
temp_max_media	DoubleType	Media nacional de las temperaturas máximas del día (°C).	AEMET
temp_min_media	DoubleType	Media nacional de las temperaturas mínimas del día (°C).	AEMET
temp_media_nacional	DoubleType	Temperatura media de España calculada para ese día (°C).	AEMET
precipitacion_media	DoubleType	Media de lluvia del día (mm). Las letras "lp" se cambiaron a 0.0.	AEMET
festividad	StringType	El nombre de la fiesta. Si no es festivo pone "Laborable/Normal".	API Calendarific
es_festivo	IntegerType	Pone un 1 si es festivo y un 0 si es un día normal.	Calculado (Spark)
dia_semana	IntegerType	Del 1 (Lunes) al 7 (Domingo).	Calculado (Spark)
mes	IntegerType	Número del mes, del 1 al 12.	Calculado (Spark)
anio	IntegerType	El año en el que estamos (de 2013 a 2026).	Calculado (Spark)
trimestre	IntegerType	El trimestre del año, del 1 al 4.	Calculado (Spark)
es_fin_de_semana	IntegerType	Pone un 1 si es sábado o domingo, y 0 de lunes a viernes.	Calculado (Spark)
rango_termico	DoubleType	La resta entre la temperatura	Calculado (Spark)

		máxima y la mínima del día. Si el clima falta por latencia, será NULL.	
demanda_lag_7	DoubleType	Lo que se consumió exactamente hace 7 días. Qité la primera semana para que no hubiera nulos.	Calculado (Spark)